Power and Energy Management

April 7th

EMILY CHEN AND MARIE YU

What we'll cover today:

Kareus: Joint Reduction of Dynamic and Static Energy in Large Model Training

TAPAS: Thermal- and Power-Aware Scheduling for LLM Inference in Cloud Platforms

Motivation:

Energy usage and thermal regulation have become one of the leading problems in the training and usage of large language models

The training of GPT-3 was reported to have consumed 1.3 GWh

LLM inference on a datacenter scale consumes even more energy after training, as it requires continuous energy usage

- Estimated to bump data center energy consumption to at least 9.1% of total US energy demand by 2030

Overview

Kareus	TAPAS
Efficient energy management for **LLM training**	Power and thermal management for continued **LLM inference**
Largely concerned with what's happening on the **kernel level**	Largely concerned with what's happening on the **data-center/warehouse scale**
Focused on optimizing energy and reducing energy waste	Focused on power usage and keeping within thermal requirements

Kareus: Joint Reduction of Dynamic and Static Energy in Large Model Training

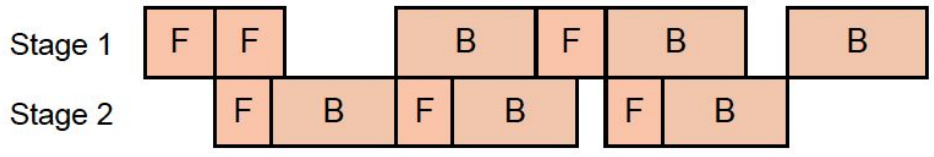
Problem

- AI demand is increasing at an unprecedented rate, and energy supply is not keeping pace
- By 2035, nearly 10% of US electricity demand could be from datacenters
- Recent works in large model training optimization focus on either **static** or **dynamic** energy consumption, not both
  - **Dynamic energy**: consumed by actual work
  - **Static energy**: consumed at all times, regardless of work

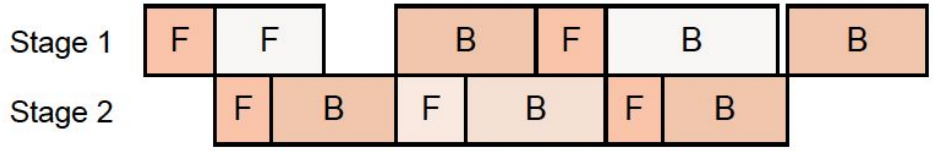
Related Works: Perseus

Perseus (Chung et al., SOSP '24): Reduce **dynamic energy** by scaling the frequency of computations off the critical path

- Microbatches on critical path directly determine how long training iteration will take
- Solution: Lower the GPU frequency for microbatches not on the path
- Caveat: Doesn't reschdule kernels, which reduces static energy by overlapping computation and communication



(a) *1F1B pipeline with no frequency scaling*



(b) *1F1B pipeline with frequency scaling by Perseus*

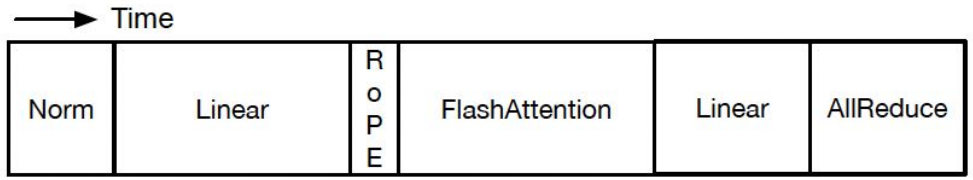
Figure 1: ***Existing training systems running the 1F1B pipeline schedule [26]. Redder colors indicate higher GPU frequency and power draw. (a) The baseline Megatron-LM [26] does not perform any frequency scaling, whereas (b) Perseus [15] scales GPU frequencies of non-critical forward and backward computations to reduce energy consumption while maintaining the same latency.***

Related Works: Fine-grained kernel scheduling

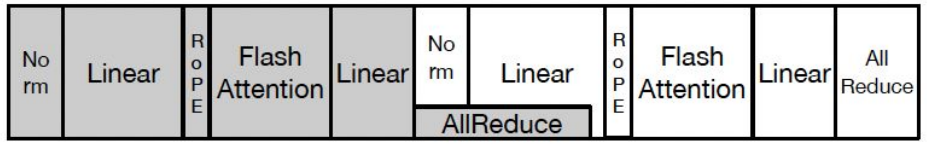
Static energy = static power x time

Nanobatching: split a microbatch into two nanobatches ⟶ Reduce static energy by reducing iteration time

- Creates opportunities to overlap communication (AllReduce) and computation kernels
- Does **not** reduce dynamic energy: GPU frequency unchanged, same amount of matrix multiplications



(a) *Sequential kernel execution*



(b) *Nanobatching*

Figure 2: ***Transformer [36] Attention layer with tensor parallelism run with different execution models. (a) The sequential kernel execution model only runs one kernel at a time strictly following data dependencies. (b) Nanobatching [17, 37, 42] splits a pipeline microbatch into two nanobatches (illustrated with different colors) and staggers their execution, creating opportunities to overlap communication and computation.***

Naive solution: Nanobatching + Perseus?

	Iteration time	Static energy	Dynamic energy	Total energy
Megatron-LM	5.60	5,372	21,374	26,745
Megatron-LM + Perseus	5.60	5,374	19,531	24,905
Nanobatching	5.31	5,096	21,445	26,541
Nanobatching + Perseus	5.37	5,160	19,729	24,889

Table 1: ***Training iteration time (seconds) and energy breakdown (Joules) of Megatron-LM, Nanobatching, and each combined with Perseus, training Qwen 3 1.7B on 16 NVIDIA A100 GPUs.***

- Nanobatching + Perseus does reduce both static and dynamic energy compared to baseline (Megatron-LM), but we can do better!

Solution: Execution Schedules

Kareus: Determine how **execution schedules** impact time and energy consumption

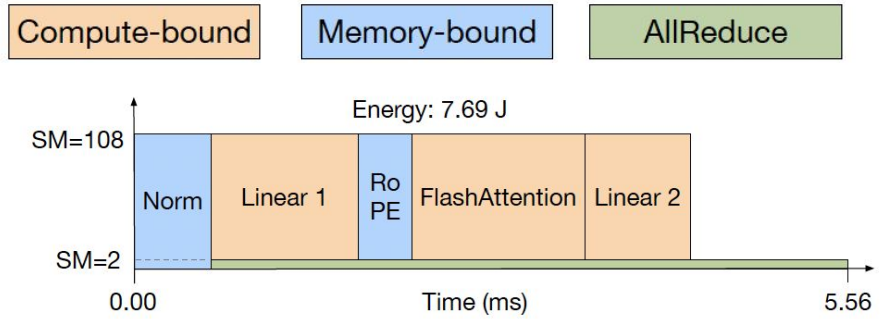
- Execution schedules are defined as the combination of three factors:
  1. **Communication kernel launch timing:** When to overlap communication kernels with computation kernels
  2. **Number of SMs** (Streaming multiprocessors) allocated to communication kernels vs. computation kernels
     a. Each SM is like a mini-processor — a NVIDIA A100 has 108 SMs
  3. **GPU Frequency**: rate at which GPU processors complete cycles of operation
     a. Dynamic Power = $k \times \text{frequency}^3$

Optimizing execution schedules can cause up to a 3.29x variation in time/energy consumption, while total work stays the same

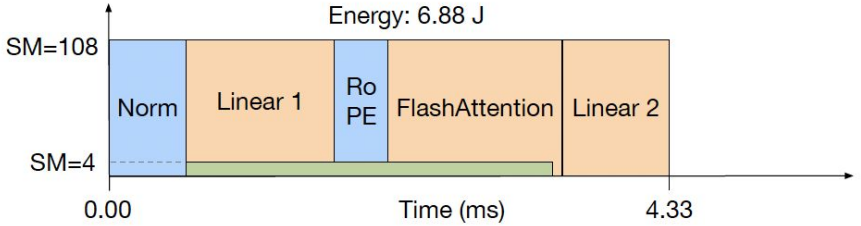
Joint Optimization: SM Allocation

Time and total energy consumption of execution schedules for one Transformer Attention layer forward pass

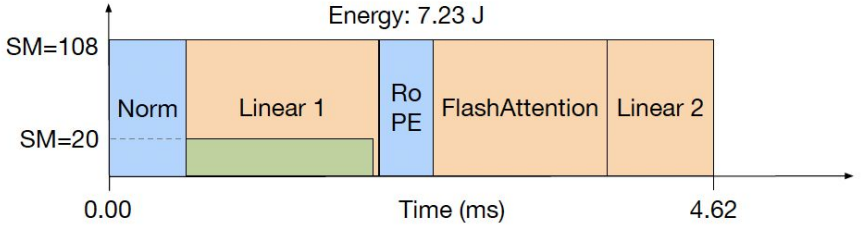
- Varying SM allocation, constant GPU frequency (1410 MHz)
- Too few SMs allocated to communication kernels⟶ idle GPU (a)
- Too many SMs allocated for communication kernels ⟶ compute interference (c)



(a) *2 SMs lead to exposed communication time*



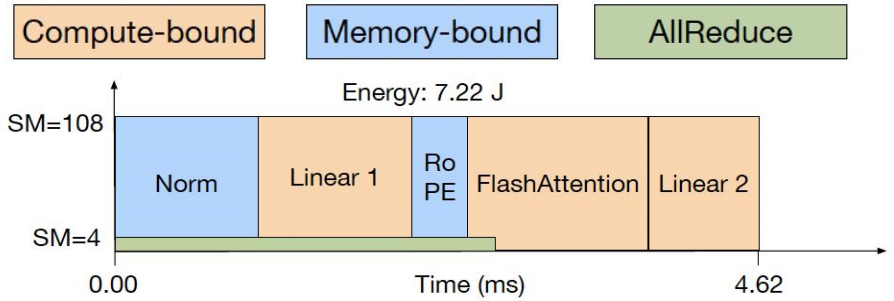
(b) *4 SMs eliminate exposed communication time*



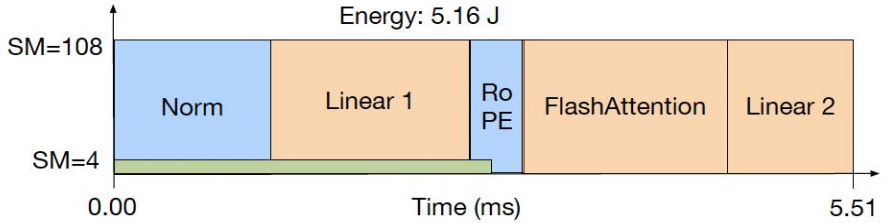
(c) *20 SMs reduce communication latency but lead to compute interference*

Communication Launch Timing + GPU Frequency

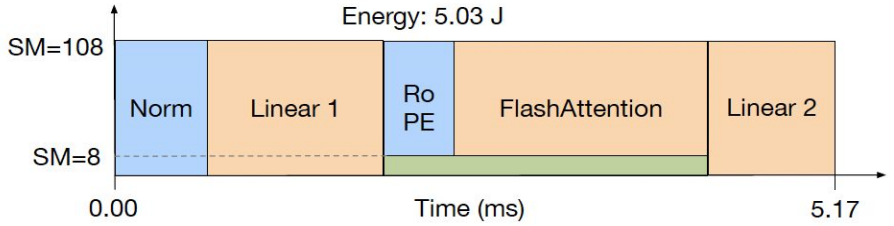
- Launching communication at the same time as Norm causes competition with Linear 1 for memory bandwidth → energy increases to 7.22J (d)
- Lowering GPU frequency alone is not enough: execution time increases because communication kernel takes SMs away from Linear 1 (e)
- Lowering GPU frequency also changes the energy-optimal schedule (f)



(d) *Same as (b) but communication was launched with* `Norm`



(e) *Same as (d) but runs at a lower GPU frequency (1,100 MHz)*



(f) *Lower frequency (1,100 MHz) changes the energy-optimal schedule*

Introducing Kareus

- Jointly optimizes all three factors: kernel timing, SM allocation, and GPU frequency
- **Goal**: Find the time-energy tradeoff frontier of executing computation and communication kernels of large model training
  1. Partition overlap model → decompose problem into tractable subproblems
  2. Derive time-energy frontier for each subproblem, then compose into global frontier

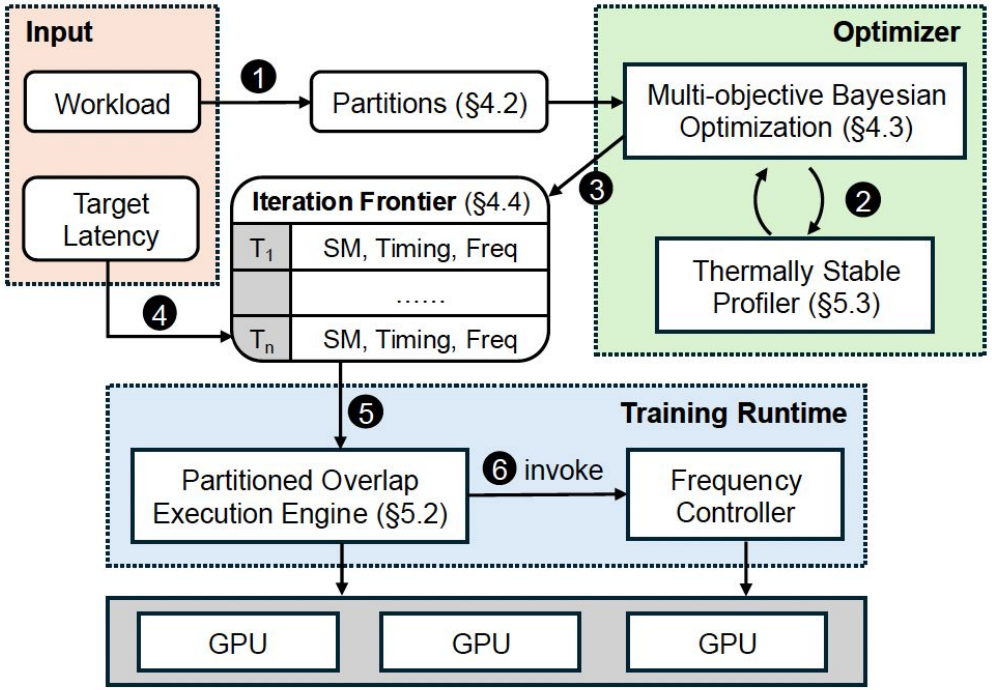
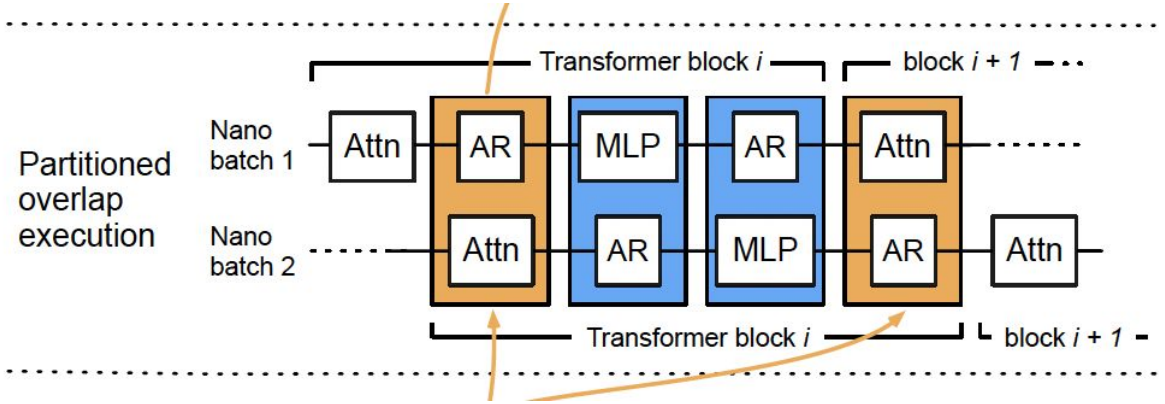


Figure 7: ***Kareus system overview.***

Partitioned Overlap Execution Model

- **Problem**: Solving global optimization problem is challenging due to large search space (85,050 candidates × 13 sec each = 307 hours)
- Instead, find repeating patterns in computation and communication sequences
- Partitions = repeating, independently optimizable units
  - One communication kernel + surrounding computation kernels with no data dependencies
- All partitions of the same type share the same configuration, making search tractable



Multi-Objective Bayesian Optimization

Goal: Find the Pareto frontier of **time** and **energy** for each partition — the set of schedules where you can't improve one without worsening the other

Surrogate Model: XGBoost (gradient-boosted decision trees)

- Trains two cheap models to predict time and energy respectively, without actually running each configuration
- Why XGBoost: scales linearly with data points, allowing fast retraining
  - Tree structure handles discrete (SM count, frequency) and categorical (launch timing) variables naturally
- Each real measurement updates the model, making predictions progressively more accurate

Multi-pass candidate selection

Expanding the frontier from all directions:

- Total energy pass: targets configurations that push the overall frontier outward
- Dynamic energy pass: hunts for lower-frequency configurations to reduce computation energy
- Static energy pass: hunts for faster configurations to reduce idle time energy
- Uncertainty (exploration) pass: explores poorly understood regions to avoid missing hidden good configurations

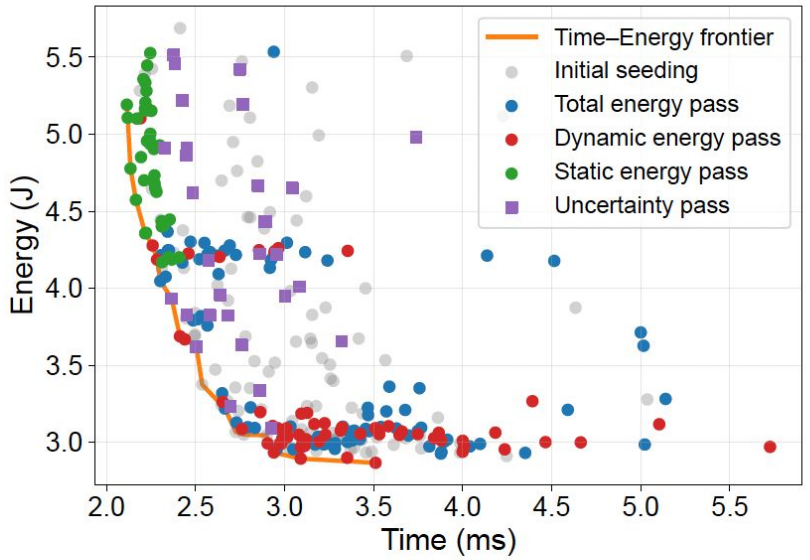


Figure 6: ***Multi-pass MBO in action with the Llama 3.2 3B model's MLP–AllReduce partition. The three exploitation passes (total energy, dynamic energy, static energy) and the exploration pass expand the efficient frontier in complementary directions, as shown by the colors of the points that land on the time–energy frontier.***

Composing Frontiers

With each partition's time-energy frontier, Kareus composes **microbatch-level** and then **iteration-level** frontiers

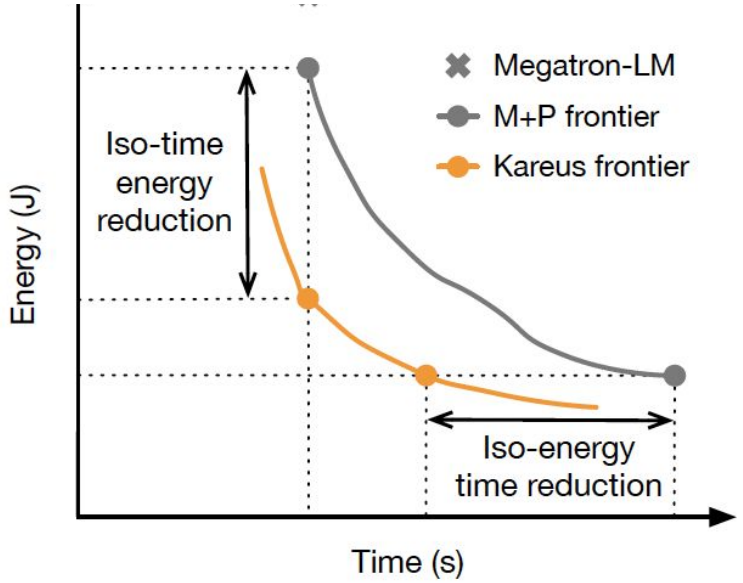
- **Partition → Microbatch frontier**: Kareus enumerates combinations of partition execution schedules, sums their time and energy, and prunes suboptimal combinations
  - Enforce uniform GPU frequency across partitions in a microbatch (switching frequencies too costly)
- **Microbatch → Iteration frontier**: adopts Perseus's iterative algorithm [15] to construct the training iteration frontier from microbatch frontiers

Results: End-to-end performance

Tested on Llama 3.2 3B and Qwen 3 1.7B on 16 NVIDIA A100 GPUs

- Max-throughput: up to 14.9% time reduction and 22.1% energy reduction vs. Megatron-LM baseline
- Frontier improvement vs. Perseus:
  - Up to 28.3% energy reduction at the same training time
  - Up to 27.5% time reduction at the same energy budget

Model	Parallelism	μBatch Size	Sequence Length
Llama 3.2 3B	TP8	8	4K
		8	8K
		16	4K
Llama 3.2 3B	CP2TP4	8	4K
		8	8K
		16	4K
Qwen 3 1.7B	TP8	8	4K
		8	8K
		16	4K
Qwen 3 1.7B	CP2TP4	8	4K
		8	8K
		16	4K



Results: Interesting Case Study

Qwen 3 1.7B, TP8, sequence length 4K

- Both baselines ran the critical-path microbatch at 1,410 MHz (max)
- Kareus discovered that running at lower frequency (1,350 MHz) actually achieved the fastest latency
  - Nanobatching at max frequency raised instantaneous power, triggering GPU throttling — causing frequency to fluctuate around 1,350 MHz but with higher dynamic energy than just running steadily at 1,350 MHz
- Illustrates **Theorem 1**: *GPU frequency locked to the average frequency of a dynamic frequency schedule consumes less total energy.*

Results: Large Scale Emulation

- Real hardware limited to 16 GPUs → emulate to verify Kareus at production scale
- Emulated Llama 3.3 70B at up to 10,240 GPUs across 4 configurations varying number of microbatches (16, 32, 64, 128)
  - Pipeline parallelism degree 10, tensor parallelism degree 8, global batch size 2,048 → matching real Llama 3 training configuration
- Kareus achieves ~20% energy reduction vs. Megatron-LM across all configurations
- Up to 15.3% iso-time energy reduction and 19.1% iso-energy time reduction vs. Perseus alone

Table 7: *[Emulation] Iteration time and energy reduction (%) relative to Megatron-LM of Megatron-LM + Perseus and Kareus under the max-throughput configuration for Llama 3.3 70B.*

# Microbatches	Time Reduction (%)		Energy Reduction (%)	
	M+P	Kareus	M+P	Kareus
16	0.0	9.3	15.0	20.2
32	0.0	9.2	14.3	20.0
64	0.0	9.1	13.8	19.8
128	0.0	9.1	13.5	19.7

Table 8: *[Emulation] Iso-time energy reduction (%) and iso-energy time reduction (%) relative to Megatron-LM + Perseus of Kareus for Llama 3.3 70B.*

	# Microbatches			
	16	32	64	128
Iso-Time Energy Reduction (%)	11.6	14.1	15.3	15.1
Iso-Energy Time Reduction (%)	19.1	16.8	16.4	16.0

Results: Overhead and Ablation

- MBO runs once before training begins and takes ~2 hours on average
  - Negligible vs weeks taken to train Llama 3 (54 days)
  - Of the 2 hours: 97% is thermally stable profiling, 3% is actual search algorithm
- All four candidate passes are necessary:
  - Random initialization alone lands on only 35% of final frontier candidates → four targeted passes discover 65% of candidates
  - Contribution breakdown of the 65%:
    - Total energy pass: 25%
    - Dynamic energy pass: 22%
    - Static energy pass: 12%
    - Uncertainty pass: 6%

Conclusions

Takeaways:

- SM allocation, kernel launch timing, and GPU frequency are interdependent
- Optimizing any subset in isolation leaves significant savings on the table
- Energy must become a first-class design constraint alongside speed and accuracy

Limitations:

- Evaluated only on NVIDIA A100 GPUs; optimal schedules on newer models like H100s may differ
- Current design assumes a fixed pipeline schedule (1F1B) — applicability to newer or more dynamic scheduling strategies is an open question

TAPAS: Thermal- and Power-Aware Scheduling for LLM Inference in Cloud Platforms

Motivation

Today, use cases for LLM inference are becoming ubiquitous

- Chatbots
- Agentic AI
- Use cases: programming, healthcare, education, e.t.c.

The demand means that LLM inference needs cloud infrastructure to serve it!

Problem

Thermal and power management for traditional datacenters has been extensively studied.

However, datacenters serving LLM inference have unique characteristics that make traditional datacenter designs sub optimal.

LLM inference becomes **expensive**!

Datacenter workload and topology that TAPAS will be studying:

- 50/50 IaaS (Infrastructure as a Service) and SaaS (Software as a Service)

Related works

Datacenter cooling management

CoolProvision: optimizes under-provisioned cooling

- warm water, immersion-cooling, free-cooling

LLM serving

POLCA: power oversubscription framework

μ-Serve enables power-aware LLM serving through model parallelism and predictive request scheduling.

- KV Cache management
- continuous batching

Thermal-aware scheduling

RT-TAS: thermally-balanced task-to-core assignment

PTDS: optimizes VM-to-host scheduling, prevent hotspots and reduce cooling energy.

Datacenter power management

Flex: safely oversubscribes reserved power

SmartOClock: distributes power budgets efficiently through power predictions for workload-aware over-clocking.

SmoothOperator: spreads services with synchronous power patterns evenly to reduce peak power draw

Challenges: Cooling

AHU: air handling units

Things to consider:

- provisioning
- failures
- outside temperature (cooling technologies use outside air when it is cold)
- datacenter layout
- datacenter load
- GPU heterogeneity
- airflow

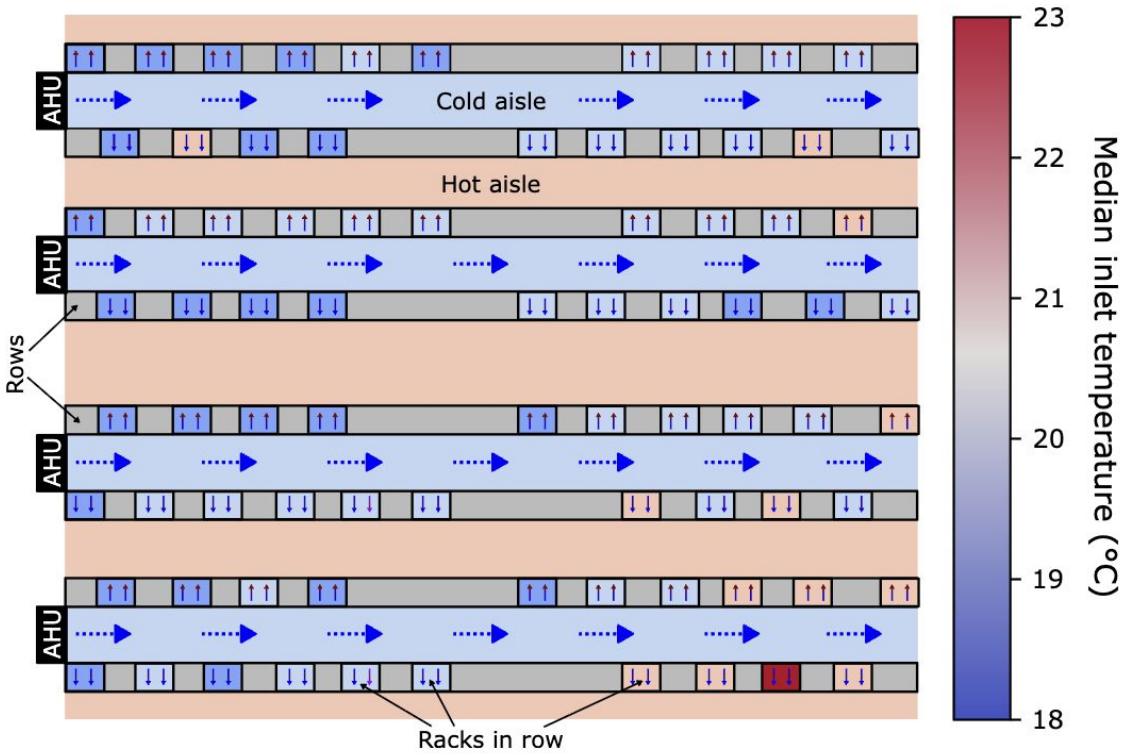


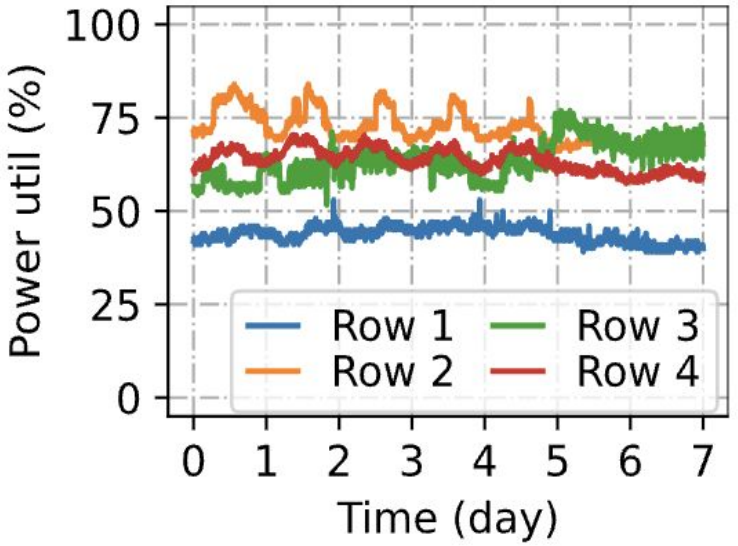
Figure 1. Sample datacenter layout illustrating 80 racks organized into 8 rows with 4 cold aisles. The rack color represents the inlet temperatures for the top server.

Challenges: Power

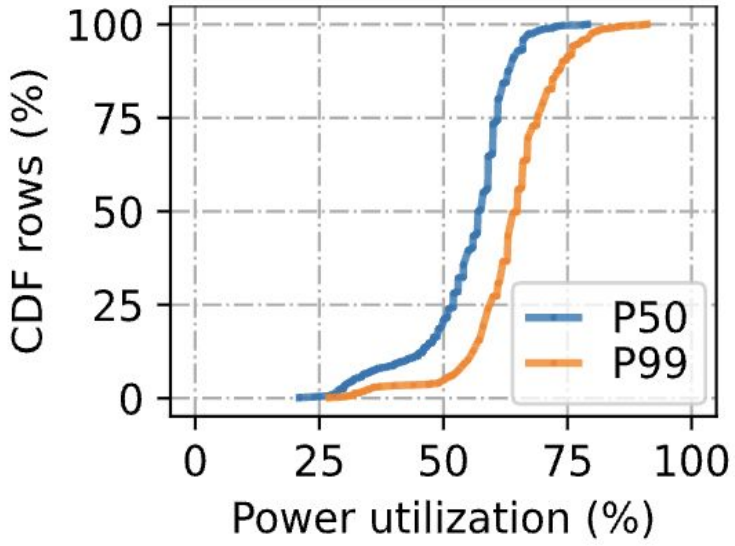
Automatic Transmission Switch (ATS) → Uninterruptible Power Supplies (UPS) → Power Distribution Unit (PDU) pairs

Things to consider:

- provisioning
- failures
- server power correlated with GPU load
- power imbalance among rows



(a) Power utilization over 1-week.



(b) Row power distribution.

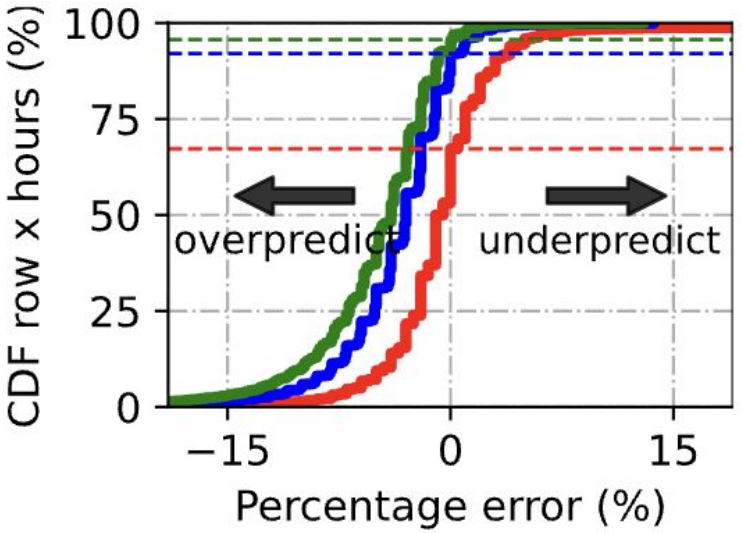
Insights into GPU workloads

VMs are typically long lived

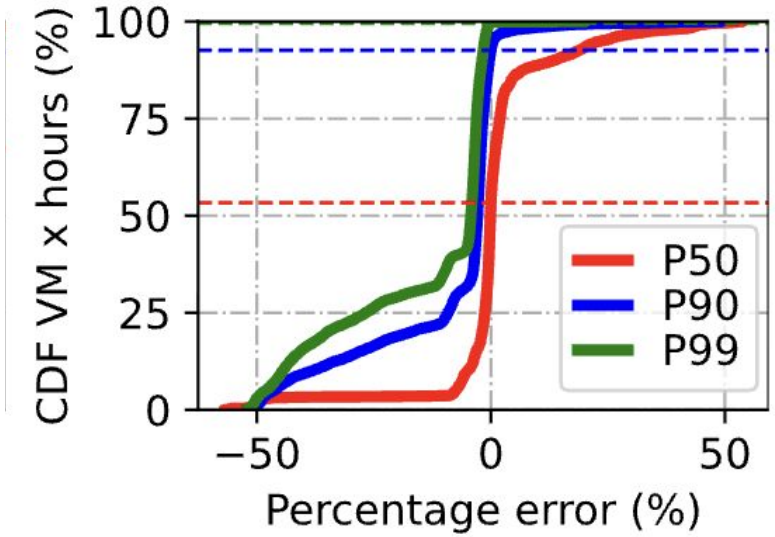
Predictable load - row power prediction using past history has less than 10% error for most row hours

Traditional VM allocation rules are thermal/power agnostic however…

- SaaS workloads are flexible and can be request routed based on thermal/power conditions



(a) Row-based.

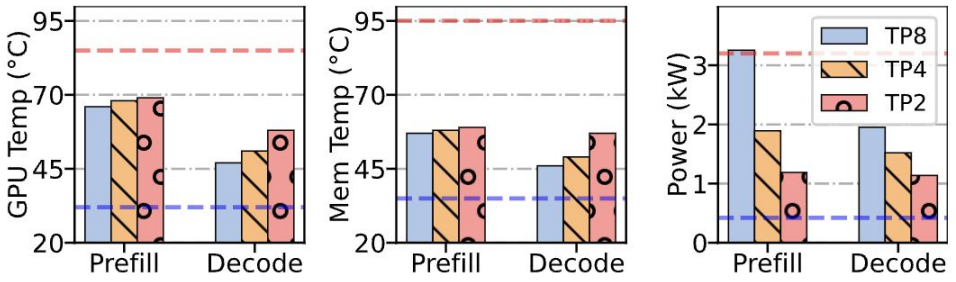


(b) Customer-based.

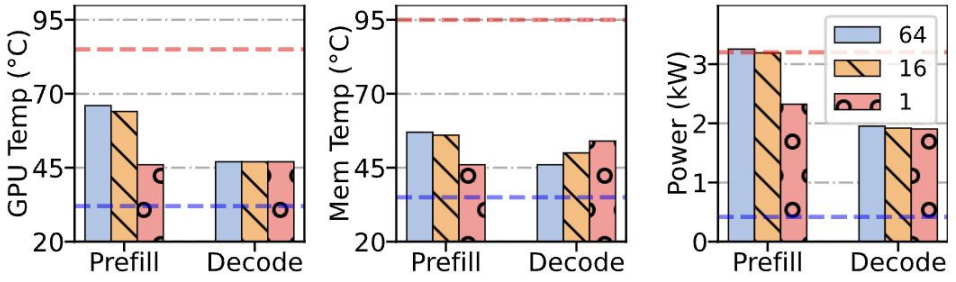
Configurable parameters

Temperature varies based on model configuration/parameters

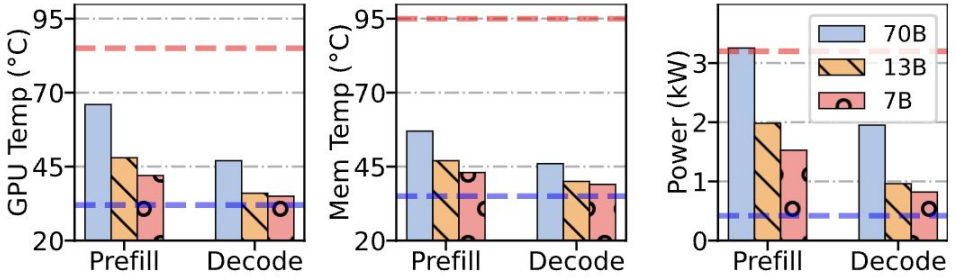
Insight: this can be leveraged to manage temperature and power, while trying to not decrease result quality



(a) Tensor parallelism [63].



(b) Batch size [81].



(c) Llama2 model sizes [72].

Solution: TAPAS

The authors:

- study the unique characteristics of datacenters with GPUs, serving primarily LLM inference workloads
- propose TAPAS, a **thermal** and **power** aware scheduler that works within these cloud datacenters

Solution: TAPAS

TAPAS:

- places GPU VMs in a thermal-aware/power-aware manner (inferred on historical data)
- routes requests based on load, thermal, and power capacity
- reconfigures SaaS to restore temperature and power usage to safe levels

Configuration parameters	Perf	Temp	Power	Quality
Model Size (*e.g.*, 70B→7B)	↑	↓	↓	↓↓
Quantization (*e.g.*, FP16→FP8)	↑	↓	↓	↓
Parallelism (*e.g.*, TP8→TP2)	↓	↑	↓	–
Frequency (*e.g.*, 2GHz→1GHz)	↓	↓	↓	–
Batch Size (*e.g.*, 64→16)	↓	↓	↓	–

Table 1. Impact of different parameters on the performance, temperature, power, and quality of an LLM inference server. TP stands for tensor parallelism.

TAPAS Architecture

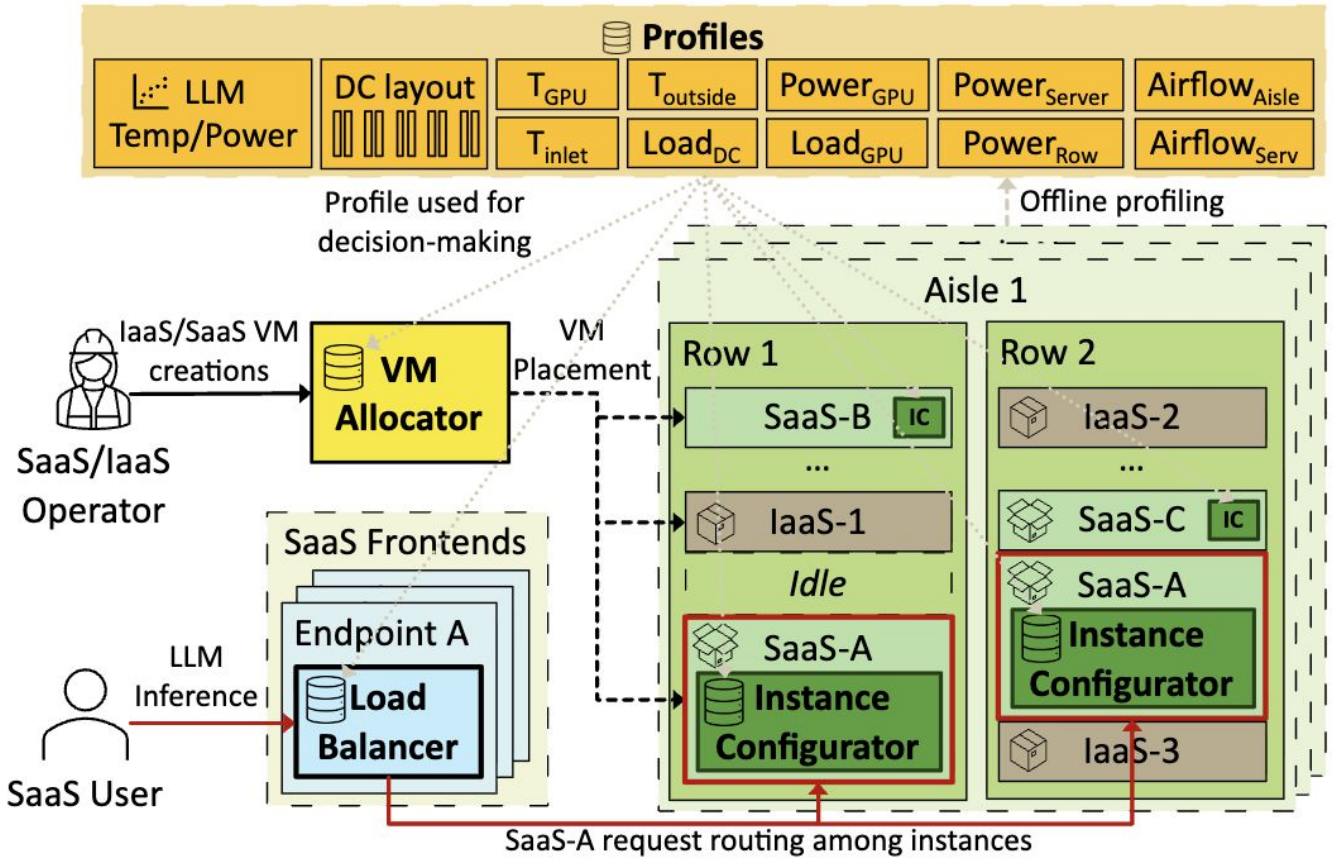


Figure 18. TAPAS architecture overview.

Workload placement

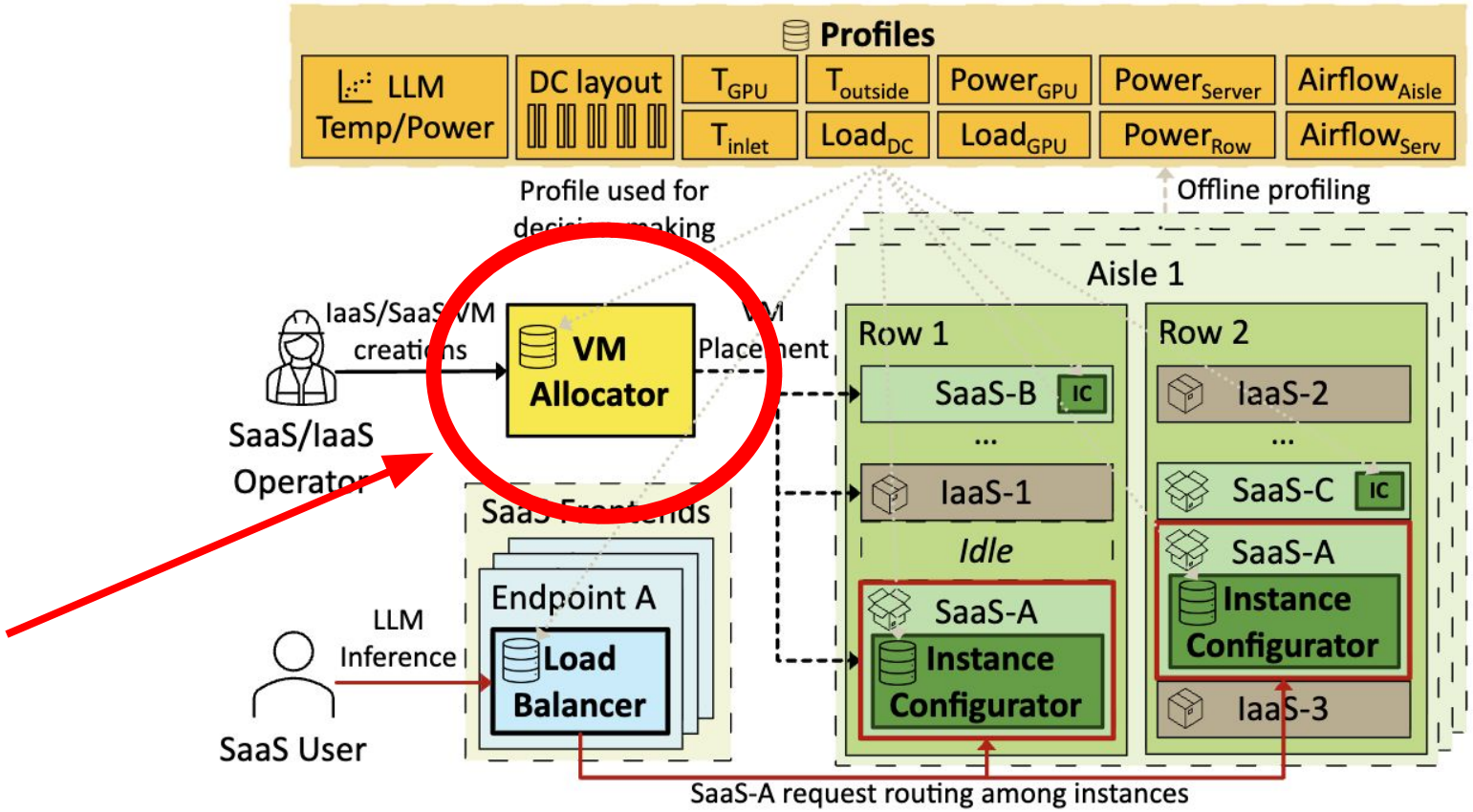


Figure 18. TAPAS architecture overview.

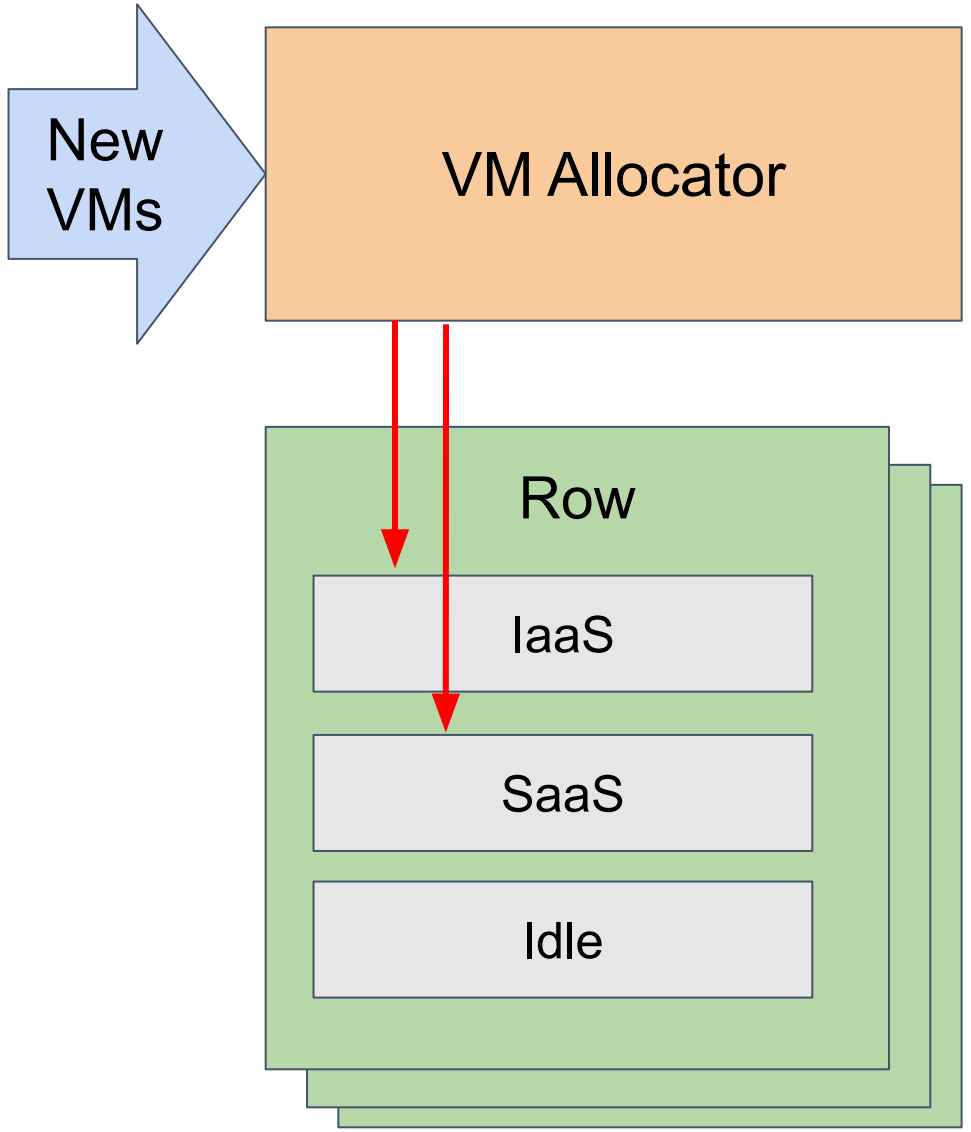
Workload placement

Filter out rows that would exceed limits if VM placed

Place IaaS VMs in cooler servers

Balance IaaS and SaaS workloads

Migration: VM migration if mispredictions occur (currently unsupported)



Request routing

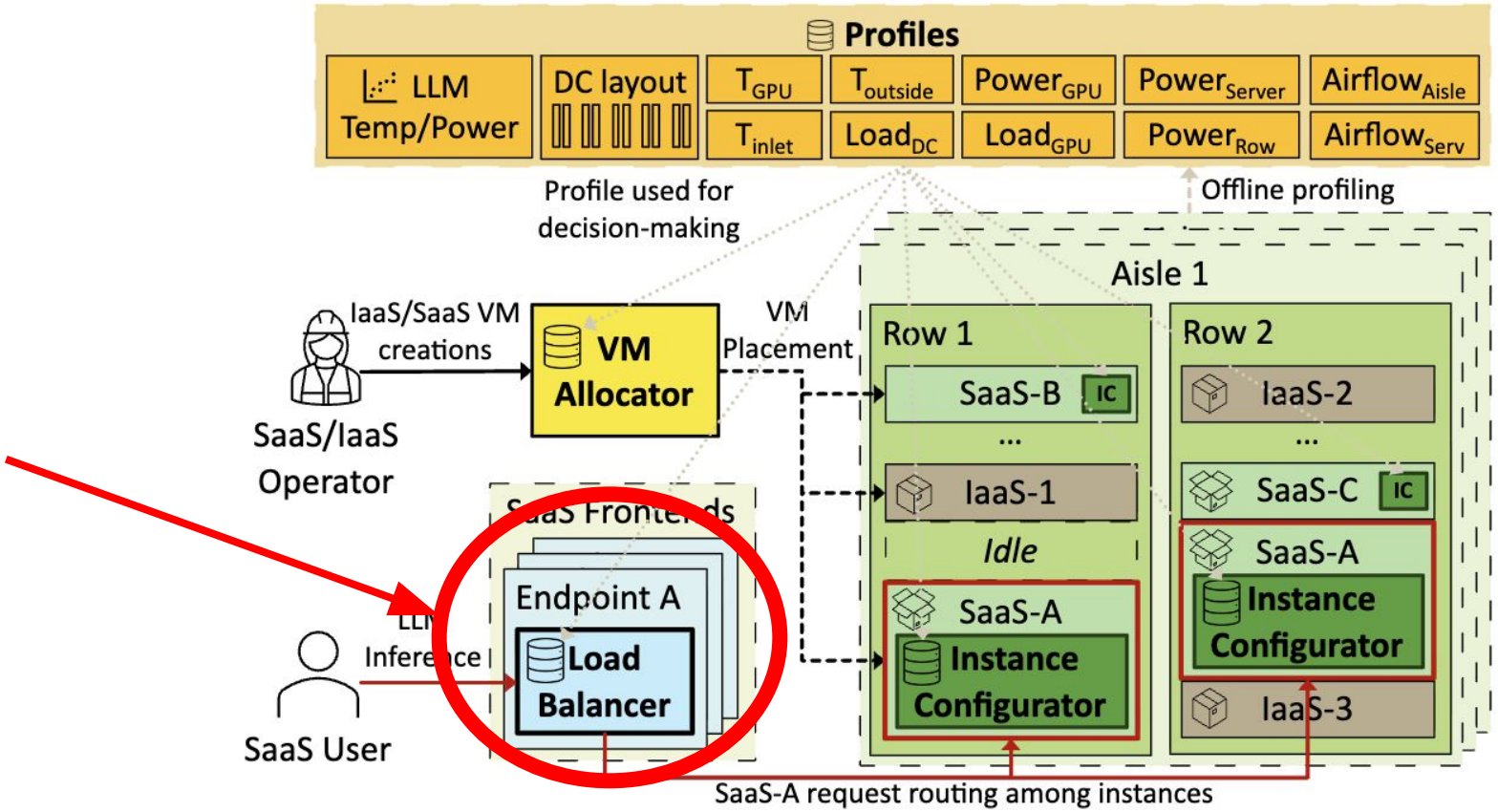


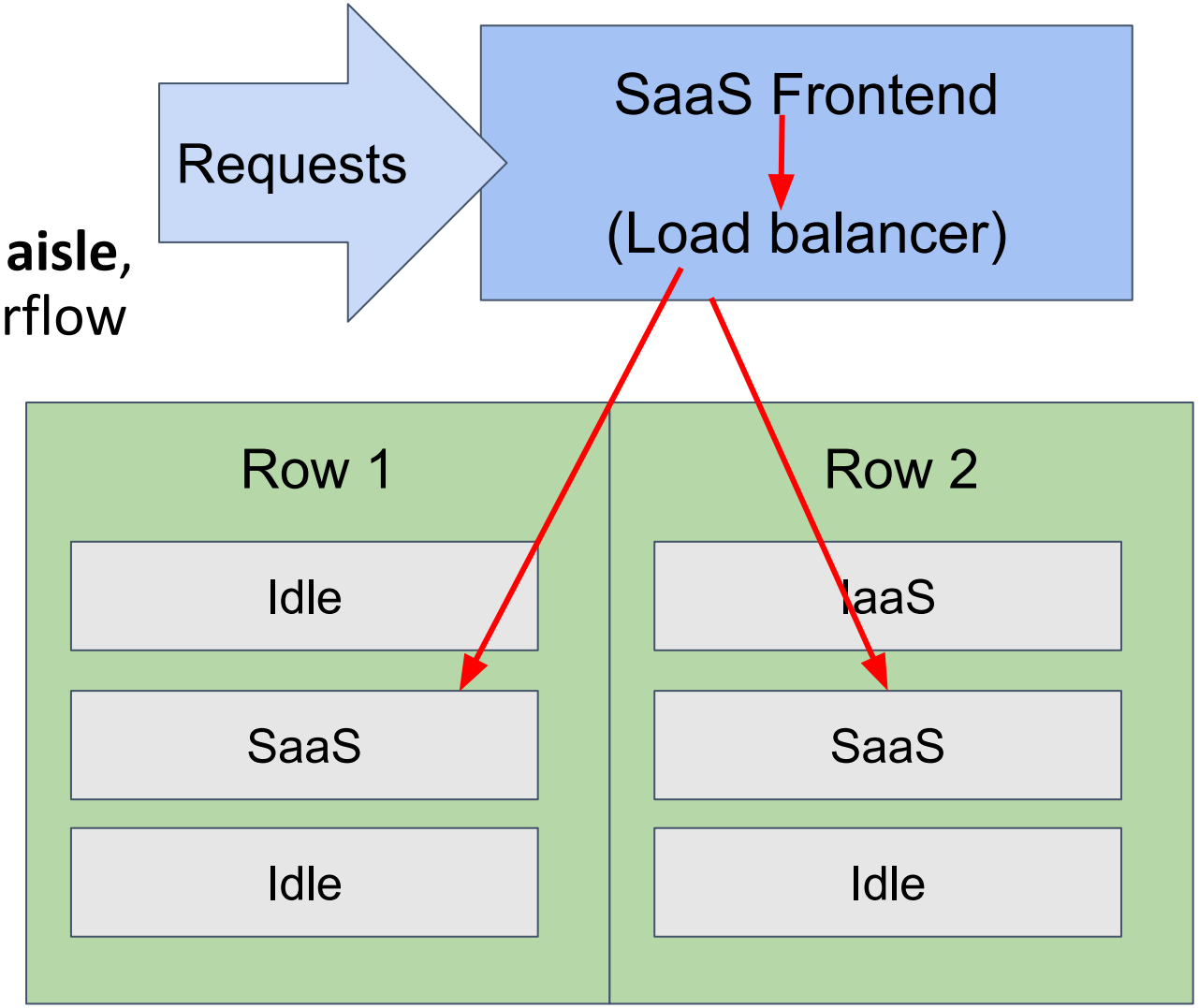
Figure 18. TAPAS architecture overview.

Request routing

Calculates total airflow demand for VMs per **aisle**, prevents routing to VMs that could trigger airflow violation

Assess whether routing requests to VM per **row** will go over the power limit

Tracks load per **server** and avoids routing requests to VMs that could overheat



Instance configuration

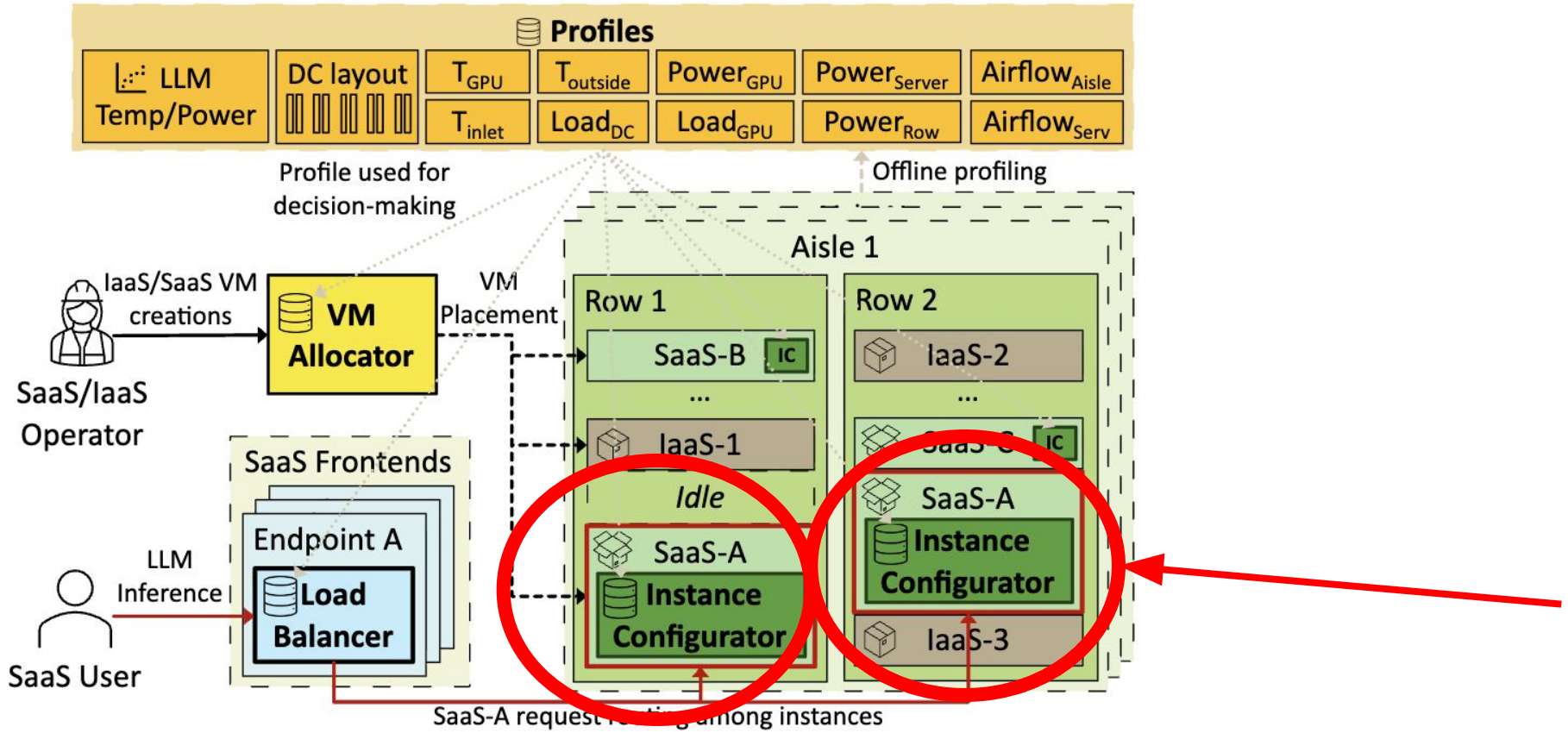


Figure 18. TAPAS architecture overview.

Instance configuration

Calculate max airflow, GPU temperature, and server power

Leverages power profiles to determine what configurations to change without impact on quality

Profiling only done when new model is registered w/ TAPAS

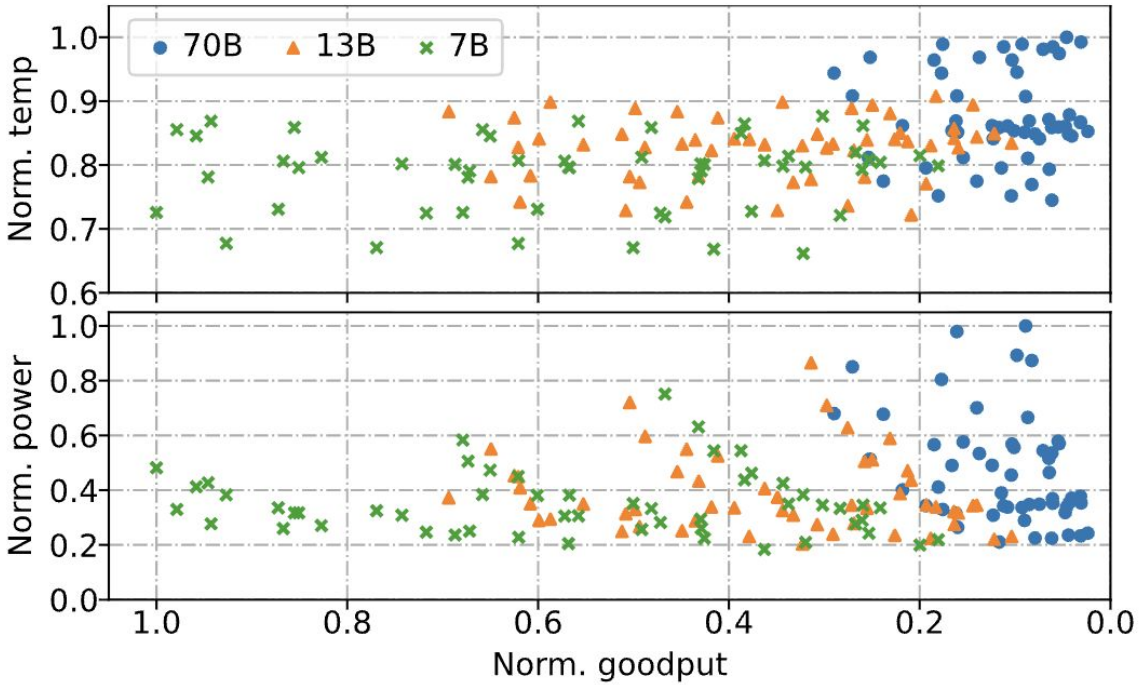


Figure 17. Normalized temperature and power (lower is better) and goodput (higher is better) of Llama2 [72] with all configuration parameters highlighting the model size.

Implementation

Offline profiling phase

Models:
- datacenter layout
- inlet temp of each server
- temp of each GPU
- server fan airflow
- server power load

Profiles LLM configurations as they get registered

Weekly refinement

Allocators/Schedulers

rule based **VM allocator**

uses a thermal and power aware **load balancer**, filter out VMs that could cause violations
- maximize cache reuse and reduce energy

uses an **instance configurator** to find optimal model configs depending on temp and power

Evaluation

Baseline -> thermal and power-oblivious, traditional VM placement and request routing, no LLM configuration

Use a cluster to **emulate** two rows of 80 A100 servers (50/50 IaaS and SaaS mix)

Built a **simulator** to model a real datacenter with factors we previously described, replicating the load of IaaS VMs and SaaS requests

- Used cooling regression models
- Uses historical power readings for IaaS

Results

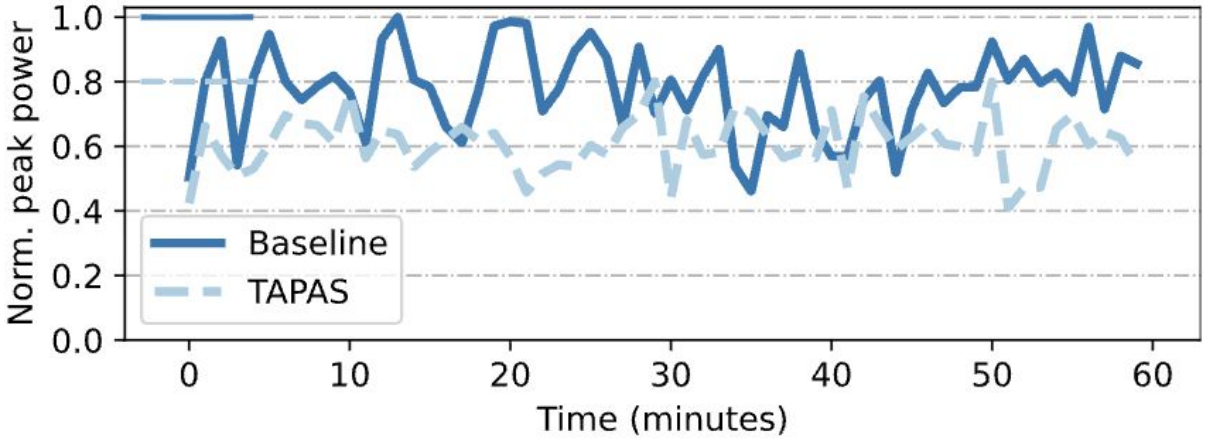


Figure 19. Peak power over 1-hour for *Baseline* and TAPAS while running real cluster experiments.

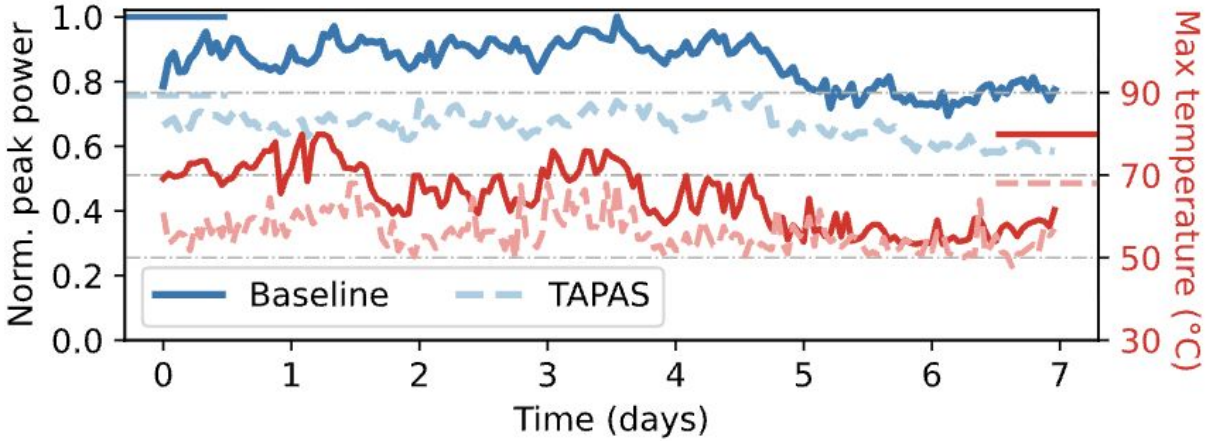


Figure 20. Peak power and maximum temperature over 1-week for *Baseline* and TAPAS for large-scale simulations.

Results

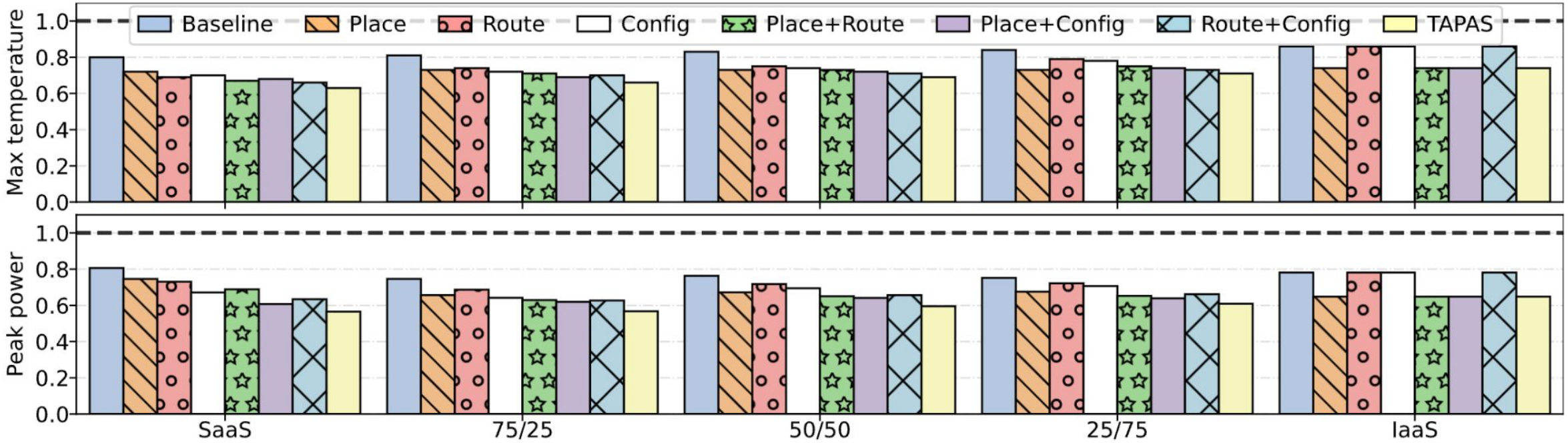


Figure 21. Normalized maximum temperature and peak power varying the policy and fraction of SaaS and IaaS workloads.

Results

	Power Emergency				Thermal Emergency			
	Baseline		TAPAS		Baseline		TAPAS	
	IaaS	SaaS	IaaS	SaaS	IaaS	SaaS	IaaS	SaaS
Perf	-35%	-28%	0%	+16%	-22%	-19%	0%	+10%
Quality	0%	0%	0%	-12%	0%	0%	0%	-6%

Table 2. Comparison of *Baseline* and TAPAS in power and thermal emergencies across IaaS and SaaS

Takeaways

Power and energy is a crucial issue in the training and usage of generative AI models!

Both works refine **scheduling** to maximize energy efficiency and GPU usage.

Kareus:

SM allocation, kernel launch timing, and GPU frequency are interdependent and must be jointly optimized

To optimize: decompose the problem and use multi-objective Bayesian optimization to search it efficiently

TAPAS:

Paying attention to temperature and power gives efficiency and performance gains

Can’t make traditional assumptions on GPU intensive workloads - dependent on GPU manufacture, workload, configuration, e.t.c

Questions